Artifact Narrative – Rescue Suitability Scoring Engine

Artifact Description

The artifact for this enhancement is the Rescue Suitability Scoring Engine added to the Grazioso Salvare Dashboard. The original dashboard only filtered animals based on fixed criteria such as breed, age, and sex. It could identify eligible animals but could not prioritize or rank them. This enhancement introduced a dedicated scoring module (rescue_scoring.py) that applies algorithmic logic, data structures, caching, and normalization to compute a weighted suitability score for each animal.

Justification for Inclusion

I selected this artifact because it demonstrates clear growth in algorithmic thinking and data-structure design. The scoring engine transforms the dashboard from a simple filter tool into a decision-support system. It showcases my ability to design composed algorithms, apply performance optimizations, and use Python's data structures effectively. This enhancement directly aligns with the Algorithms & Data Structures outcome and demonstrates my ability to solve real-world analytical problems.

Key Enhancements Completed

1. Weighted Scoring Algorithm I designed a multi-factor scoring system that evaluates each animal based on breed compatibility, age suitability, and outcome history.

Breed scoring: Uses a dictionary of weighted keywords (e.g., "labrador": 10, "shepherd": 10).

Age scoring: Ideal working range (26–156 weeks) adds +10; very old animals (>300 weeks) subtract −5.

Outcome scoring: Adoption adds +5; transfer adds +3; euthanasia subtracts −10.

2. Breed Normalization with Regex + Caching Mixed breed strings such as "Labrador Retriever Mix" are normalized using a compiled regex and an LRU cache.

Before: "Labrador////Mix" produced inconsistent results.

After: Normalization consistently returns ["labrador", "retriever"], and repeated values are cached for performance.

3. Ranking Algorithm Animals are ranked by descending rescue score. The dashboard displays the ranked list and includes the score in both the table and the map info card.

Testing Evidence

Testing demonstrated that the scoring engine behaves correctly across normal and edge cases:

Normal scoring results:

- *WaterRescueDog* ("Labrador Retriever Mix", 52 weeks, Adoption) → 44 points
- *MountainRescueDog* ("German Shepherd", 80 weeks, Transfer) → 23 points
- *SeniorDog* ("Newfoundland", 320 weeks, Adoption) → 8 points

These results confirm that breed weights, age penalties, and outcome bonuses work as intended.

Edge-case results:

- Missing breed → score = age + outcome only
- Missing age → score = breed + outcome only
- Missing outcome → score = breed + age only
- Negative age → score = 0 (invalid age ignored)
- "MixMixMixMix" → normalized to [], score = 0
- Identical scores → ranking preserves stable ordering

These tests demonstrate robustness and predictable behavior even with messy real-world data.

Reflection on the Enhancement Process

Enhancing this artifact taught me how to design algorithms that balance clarity, performance, and real-world constraints. Choosing scoring weights required domain reasoning, not just coding decisions. Integrating the scoring engine into the dashboard required careful threading of new fields without breaking existing callbacks. The biggest challenge was ensuring that normalization and scoring behaved consistently across

unusual breed strings and incomplete records. Caching and defensive programming helped deal with these challenges.